Sec 5.3. Tcpdump

Tcpdump

Task 1 - Introduction

`tcpdump` is a command-line network packet analyzer commonly used on Linux, macOS, and Unix-like systems. It captures and displays network traffic flowing through a network interface in real time. The Tcpdump tool and its `libpcap` library are written in C and C++ . **tcpdump**, a packet capture tool, to sniff network traffic.

> What is the name of the library that is associated with tcpdump? libpcap

Task 2 - Basic Packet Capture

A command such as `ip address show` (or merely `ip a s`) would list the available network interfaces. In the terminal below, we see one network card, `ens5`, in addition to the loopback address.

```
user@TryHackMe$ ip a s
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state U
NKNOWN group default qlen 1000
[...]
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq
state UP group default qlen 1000
[...]
```

Network interface to listen to using `-i INTERFACE` the number of packets to capture by specifying the count using `-c COUNT`. `-n` argument such as `80` being resolved to `http`, you can use the `-nn` to stop both DNS and port number lookups. `-v` to produce a slightly more verbose output. The `-vv` will produce more verbose output; the `-vvv` will provide even more verbosity.

```
user@TryHackMe$ sudo tcpdump -i ens5 -c 5 -n
tcpdump: verbose output suppressed, use -v or -vv for full pr
```

```
otocol decode
listening on ens5, link-type EN10MB (Ethernet), capture size
262144 bytes
08:55:18.989213 IP 10.10.117.2.22 > 10.11.81.126.53378: Flags
[P.], seq 2888580014:2888580210, ack 771262362, win 922, opti
ons [nop,nop,TS val 3216251159 ecr 33295823], length 196
08:55:18.989446 IP 10.10.117.2.22 > 10.11.81.126.53378: Flags
[P.], seq 196:42
```

tcpdump: Starts the packet analyzer (captures and displays network packets).
-i ens5: Selects the network interface to listen on.
ens5 is the interface name (common in Linux cloud servers like AWS).
-c 5: Capture only 5 packets and then stop.
-n Do not resolve hostnames or service names. Shows raw IP addresses instead of DNS names.
Shows port numbers instead of service names (e.g., 443 instead of https).

Captures 5 raw network packets on interface ens5 and prints them using IP addresses/ports only.

Command	Explanation
`tcpdump -i INTERFACE`	Captures packets on a specific network interface
`tcpdump -w FILE`	Writes captured packets to a file
`tcpdump -r FILE`	Reads captured packets from a file
`tcpdump -c COUNT`	Captures a specific number of packets
`tcpdump -n`	Don’t resolve IP addresses
`tcpdump -nn`	Don’t resolve IP addresses and don’t resolve protocol numbers
`tcpdump -v`	Verbose display; verbosity can be increased with `-vv` and `-vvv`

Consider the following examples:

- `tcpdump -i eth0 -c 50 -v` captures and displays 50 packets by listening on the `eth0` interface, which is a wired Ethernet, and displays them verbosely.

- `tcpdump -i wlo1 -w data.pcap` captures packets by listening on the `wlo1` interface (the WiFi interface) and writes the packets to `data.pcap`. It will continue till the user interrupts the capture by pressing CTRL-C.
- `tcpdump -i any -nn` captures packets on all interfaces and displays them on screen without domain name or protocol resolution.

Answer the questions below

> What option can you add to your command to display addresses only in numeric format? -n

Task 3 - Filtering Expressions

Although you can run `tcpdump` without providing any filtering expressions, this won't be useful. You can easily limit the captured packets to this host using `host IP` or `host HOSTNAME`. In the terminal below, we capture all the packets exchanged with `example.com` and save them to `http.pcap`. It is important to note that capturing packets requires you to be logged-in as `root` or to use `sudo`.

```
user@TryHackMe$ sudo tcpdump host example.com -w http.pcap
tcpdump: verbose output suppressed, use -v[v]... for full pro
tocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot leng
th 262144 bytes
16:49:02.482295 IP 192.168.139.132.49480 > 93.184.215.14.htt
p: Flags [S], seq 3330895816, win 32120, options [mss 1460,sa
ckOK,TS val 621343956 ecr 0,nop,wscale 7], length 0
```

If you want to limit the packets to those from a particular source IP address or hostname, you must use `src host IP` or `src host HOSTNAME`. Similarly, you can limit packets to those sent to a specific destination using `dst host IP` or `dst host HOSTNAME`. If you want to capture all DNS traffic, you can limit the captured packets to those on `port 53`. Remember that DNS uses UDP and TCP ports 53 by default. You can limit your packet capture to a specific protocol; examples include: `ip`, `ip6`, `udp`, `tcp`, and `icmp`. In the example below, we limit our packet capture to ICMP packets. We can see an ICMP echo request and reply, which is a

possible indication that someone is running the `ping` command. There is also an ICMP time exceeded; this might be due to running the `traceroute` command (as explained in the Networking Essentials room). Three logical operators that can be handy:

- `and`: Captures packets where both conditions are true. For example, `tcpdump host 1.1.1.1 and tcp` captures `tcp` traffic with `host 1.1.1.1`.
- `or`: Captures packets when either one of the conditions is true. For instance, `tcpdump udp or icmp` captures UDP or ICMP traffic.
- `not`: Captures packets when the condition is not true. For example, `tcpdump not tcp` captures all packets except TCP segments; we expect to find UDP, ICMP, and ARP packets among the results.

Command	Explanation
`tcpdump host IP` or `tcpdump host HOSTNAME`	Filters packets by IP address or hostname
`tcpdump src host IP` or	Filters packets by a specific source host
`tcpdump dst host IP`	Filters packets by a specific destination host
`tcpdump port PORT_NUMBER`	Filters packets by port number
`tcpdump src port PORT_NUMBER`	Filters packets by the specified source port number
`tcpdump dst port PORT_NUMBER`	Filters packets by the specified destination port number
`tcpdump PROTOCOL`	Filters packets by protocol; examples include `ip`, `ip6`, and `icmp`

Consider the following examples:

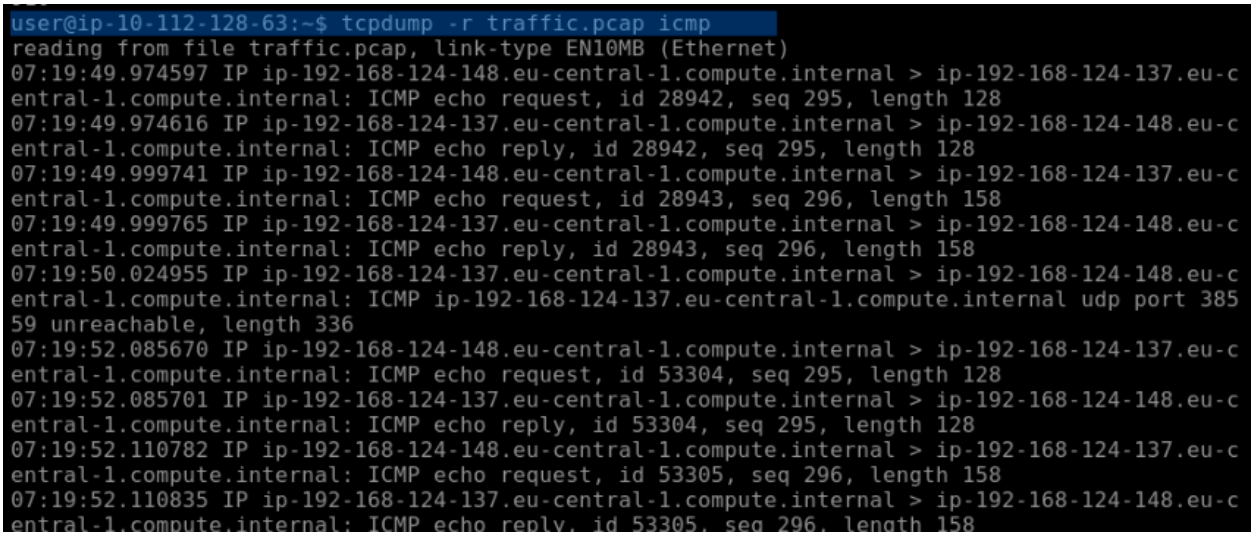
- `tcpdump -i any tcp port 22` listens on all interfaces and captures `tcp` packets to or from `port 22`, i.e., SSH traffic.
- `tcpdump -i wlo1 udp port 123` listens on the WiFi network card and filters `udp` traffic to `port 123`, the Network Time Protocol (NTP).
- `tcpdump -i eth0 host example.com and tcp port 443 -w https.pcap` will listen on `eth0`, the wired Ethernet interface and filter traffic exchanged with `example.com` that

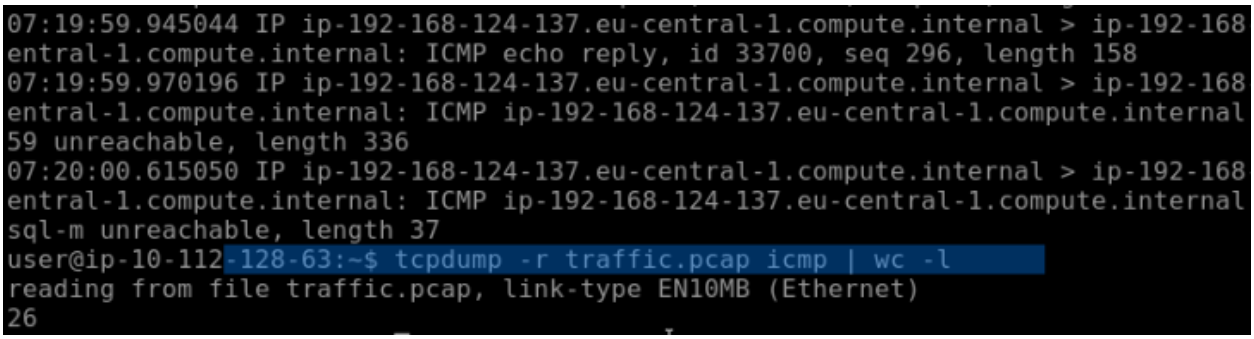
uses `tcp` and `port 443`. In other words, this command is filtering HTTPS traffic related to `example.com`.

For the questions from this task, we will read captured packets from the `traffic.pcap` file. As mentioned earlier, we use `-r FILE` to read from a packet capture file. To test this, try `tcpdump -r traffic.pcap -c 5 -n`; it should display the first five packets in the file without looking up the IP addresses. Remember that you can count the lines by piping the output via the `wc` command. In the terminal below, we can see that we have 910 packets with the source IP address set to `192.168.124.1`. Please note that we add `-n` to avoid unnecessary delays in attempting to resolve IP addresses. In the example below, we didn't use `sudo` as reading from a packet capture file does not require `root` privileges.

Answer the questions below

> How many packets in traffic.pcap use the ICMP protocol? 26

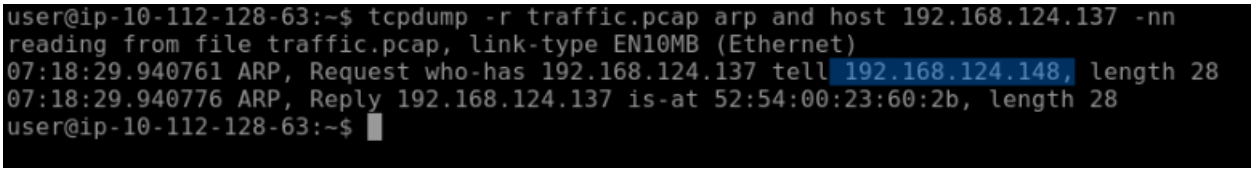




1. tcpdump -r traffic.pcap
   -r = read from file (instead of live capture)
   traffic.pcap = packet capture file
   So this part reads an existing PCAP file and prints packets.
2. icmp
   This is a filter expression
   It tells tcpdump to show only ICMP packets
   ICMP = used for tools like ping, traceroute, etc. Read the file and display only ICMP traffic”
3. | wc -l
   Pipes output into wc (word count tool)
   -l = count lines, how many lines of output tcpdump produced

“Open the PCAP file, filter only ICMP traffic, and count how many ICMP packets are there.”

> What is the IP address of the host that asked for the MAC address of 192.168.124.137? 192.168.124.148

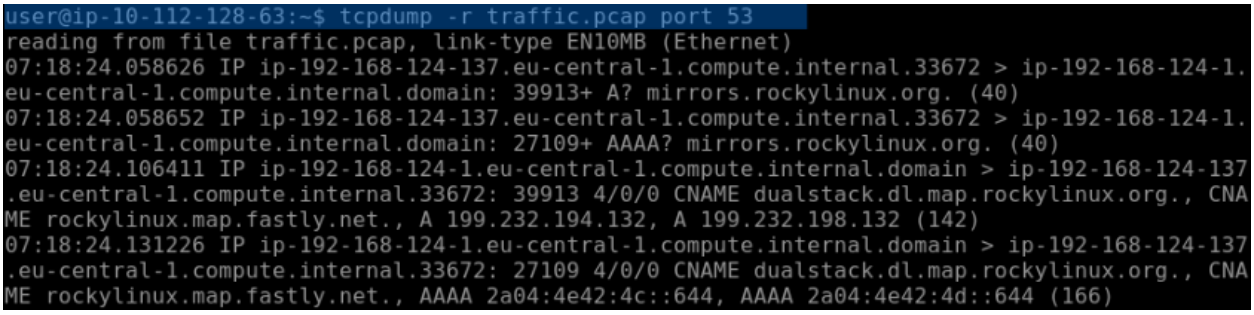


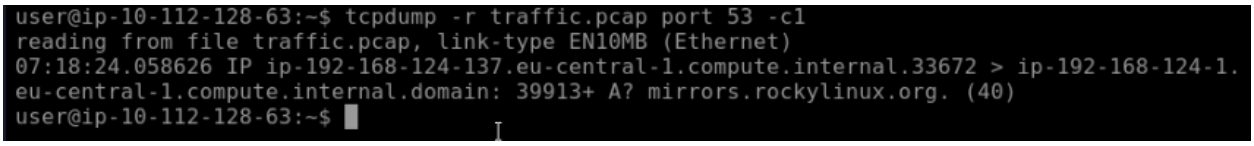
1. tcpdump Runs the packet analyzer.
2. r traffic.pcap Reads packets from the file traffic.pcap
3. arp: Filters only ARP (Address Resolution Protocol) traffic
   ARP is used to map: IP address → MAC address
   Example: “Who has 192.168.124.137? Tell me your MAC.”
4. and host 192.168.124.137
   Further filters traffic involving this IP:
   Either source OR destination is 192.168.124.137 . ARP packets where that specific host is involved
5. nn
   -n = don’t resolve DNS names (show IPs only)

-nn = also don’t resolve service names (ports)
Even though ARP doesn’t use ports, this keeps output fully numeric and clean

“Read the PCAP file and show all ARP traffic involving 192.168.124.137, without resolving names.”

> What hostname (subdomain) appears in the first DNS query?
> mirrors.rockylinux.org





port 53, Filters traffic using TCP or UDP port 53, Port 53 = DNS (Domain Name System)

So this targets: DNS queries (UDP/53), DNS responses (UDP/53), Sometimes TCP DNS (large responses, zone transfers)
-c1 Capture/display only 1 packet Then stop immediately
“From the PCAP file, show the first DNS packet (port 53 traffic) and stop.”

Task 4 - Advanced Filtering

- `greater LENGTH`: Filters packets that have a length greater than or equal to the specified length
- `less LENGTH`: Filters packets that have a length less than or equal to the specified length
- `proto` refers to the protocol. For example, `arp`, `ether`, `icmp`, `ip`, `ip6`, `tcp`, and `udp` refer to ARP, Ethernet, ICMP, IPv4, IPv6, TCP, and UDP respectively.

- `expr` indicates the byte offset, where `0` refers to the first byte.
- `size` indicates the number of bytes that interest us, which can be one, two, or four. It is optional and is one by default.

You can use `tcp[tcpflags]` to refer to the TCP flags field. The following TCP flags are available to compare with:

- `tcp-syn` TCP SYN (Synchronize)
- `tcp-ack` TCP ACK (Acknowledge)
- `tcp-fin` TCP FIN (Finish)
- `tcp-rst` TCP RST (Reset)
- `tcp-push` TCP Push

Based on the above, we can write:

- `tcpdump "tcp[tcpflags] == tcp-syn"` to capture TCP packets with **only** the SYN (Synchronize) flag set, while all the other flags are unset.
- `tcpdump "tcp[tcpflags] & tcp-syn != 0"` to capture TCP packets with **at least** the SYN (Synchronize) flag set.
- `tcpdump "tcp[tcpflags] & (tcp-syn|tcp-ack) != 0"` to capture TCP packets with **at least** the SYN (Synchronize) **or** ACK (Acknowledge) flags set.

Answer the questions below

> How many packets have only the TCP Reset (RST) flag set? 57

```
user@ip-10-112-128-63:~$ tcpdump -r traffic.pcap  "tcp[tcpflags] == tcp-rst" -nn | wc -l
reading from file traffic.pcap, link-type EN10MB (Ethernet)
57
user@ip-10-112-128-63:~$
```

tcpdump -r traffic.pcap , Reads packets from the file traffic.pcap, Offline analysis (no live sniffing)

"tcp[tcpflags] == tcp-rst" This is a BPF (Berkeley Packet Filter) expression.

tcp[...] → looks inside TCP header fields
tcpflags → TCP control flags field
tcp-rst → the RST (Reset) flag

A RST (Reset) means: The connection was abruptly terminated, One side refused or aborted communication.

-nn
-n = don’t resolve hostnames
-nn = also don’t resolve service names (ports)
Keeps output raw and faster
| wc -l Counts the number of lines in output, Each TCP RST packet usually prints as a line
“Count how many TCP reset packets exist in this PCAP file.”

> What is the IP address of the host that sent packets larger than 15000 bytes?
> 185.117.80.53

```
user@ip-10-112-128-63:~$ tcpdump -r traffic.pcap  "ip[2:2] > 15000" -nn
reading from file traffic.pcap, link-type EN10MB (Ethernet)
07:18:24.967023 IP 185.117.80.53.80 > 192.168.124.137.60518: Flags [.], seq 2140876081:214089
6901, ack 741991605, win 235, options [nop,nop,TS val 2226566282 ecr 3054280184], length 2082
0: HTTP
07:18:25.778012 IP 185.117.80.53.80 > 192.168.124.137.60518: Flags [.], seq 1293616:1308884,
ack 1, win 235, options [nop,nop,TS val 2226567095 ecr 3054280994], length 15268: HTTP
07:18:25.861724 IP 185.117.80.53.80 > 192.168.124.137.60518: Flags [.], seq 1378284:1397716,
ack 1, win 235, options [nop,nop,TS val 2226567176 ecr 3054281078], length 19432: HTTP
07:18:26.457422 IP 185.117.80.53.80 > 192.168.124.137.60518: Flags [.], seq 2356824:2373480,
ack 1, win 235, options [nop,nop,TS val 2226567777 ecr 3054281682], length 16656: HTTP
07:18:26.746414 IP 185.117.80.53.80 > 192.168.124.137.60492: Flags [.], seq 964376218:9643956
50, ack 1034282473, win 235, options [nop,nop,TS val 2226568063 ecr 3054281963], length 19432
: HTTP
```

ip[2:2]
ip = inspect the IPv4 header.
[2:2] = read 2 bytes starting at offset 2 in the IP header.
In IPv4, bytes 2–3 contain the Total Length field.

Task 5 - Displaying Packets

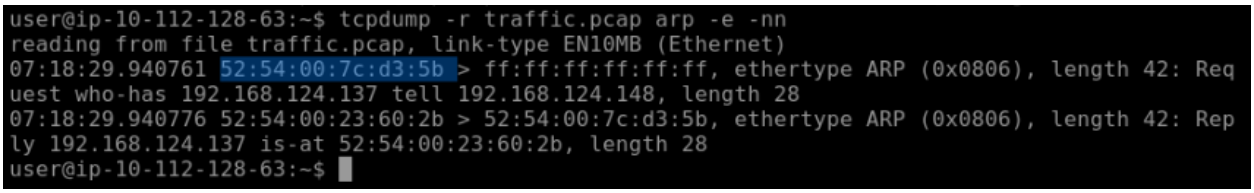
Tcpdump is a rich program with many options to customize how the packets are printed and displayed. We have selected to cover the following five options:

Command	Explanation
`tcpdump -q`	Quick and quite: brief packet information
`tcpdump -e`	Include MAC addresses

Command	Explanation
`tcpdump -A`	Print packets as ASCII encoding
`tcpdump -xx`	Display packets in hexadecimal format
`tcpdump -X`	Show packets in both hexadecimal and ASCII formats

Answer the questions below

> What is the MAC address of the host that sent an ARP request?
> 52:54:00:7c:d3:5b



arp: Filters only ARP (Address Resolution Protocol) traffic
ARP is used to discover the MAC address associated with an IP address on a local network

-e: Displays the Ethernet header for each packet, including: Source MAC address, Destination MAC address, Ethernet type, Without -e, tcpdump normally shows only higher-layer protocol information.